

SABLI: Segmented Adaptive Bloom-LSM Inverted Index for Fast Search over Unsorted Schema-less JSON Documents

whitespaca
Algorithm Design Paper

July 5, 2026

Abstract

This paper presents SABLI, a Segmented Adaptive Bloom-LSM Inverted Index for fast predicate search over unsorted schema-less JSON document collections. SABLI combines three complementary ideas: a path-value inverted index for JSON query semantics, an LSM-style segment lifecycle for write-optimized persistence and update management, and Bloom-filter-based negative pruning for avoiding unnecessary segment and block reads. Path-value indexes are effective for nested JSON predicates, but naive in-memory designs suffer from high update cost, high-cardinality fields, long cold-start time, and large memory overhead. LSM Trees provide append-friendly persistence and compaction but do not directly solve ad hoc JSON path queries. Bloom Filters efficiently reject absent terms but cannot return result sets and admit false positives. SABLI integrates these ideas into a single architecture: logical query evaluation is performed by adaptive JSON postings, storage and mutation are managed through immutable segments and compaction, and Bloom Filters only participate in access pruning. The paper defines the data model, query model, extraction function, segment model, index layout, insertion/search/update/delete algorithms, correctness properties, complexity bounds, and implementation considerations for large-scale NoSQL-like JSON search.

Keywords: NoSQL, JSON indexing, LSM Tree, Bloom Filter, inverted index, bitmap index, schema-less data, query processing, segment compaction

Contents

1	Introduction	4
1.1	Contributions	4
2	Problem Definition	5
2.1	Document Model	5
2.2	Query Model	5
2.3	Operational Constraints	5
3	Background and Motivation	6
3.1	Path-Value Inverted Indexes	6
3.2	LSM Trees	6
3.3	Bloom Filters	6
4	SABLI Architecture	7
4.1	Segment Layout	7
4.2	Logical Components	8
5	Mathematical Model	8
5.1	Extraction Function	8
5.2	Term Encoding	8
5.3	Segmented Document Space	8
5.4	Posting Function	9
5.5	Bloom Candidate Segments	9
5.6	Candidate and Answer Sets	9
6	Adaptive Index Selection	9
6.1	Cardinality Statistics	9
6.2	Representation Policy	10
7	Algorithms	10
7.1	Insertion	10
7.2	Flush	11
7.3	Search	11
7.4	Update	12
7.5	Delete	12
7.6	Compaction	12
8	Correctness	12
8.1	Bloom Filters Do Not Affect Soundness	12
8.2	Candidate Superset Property	13
8.3	Final Exactness	13
9	Complexity Analysis	13
9.1	Insertion Cost	13
9.2	Search Cost	14
9.3	Range Cost	14
9.4	Space Cost	14
10	Comparison with Component Techniques	14

11 Implementation Considerations	14
11.1 Cold Start	14
11.2 Posting Cache	15
11.3 Dictionary Encoding	15
11.4 Array Scope Semantics	15
11.5 Compaction Policy	15
12 Evaluation Plan	16
13 Limitations	16
14 Conclusion	16

1 Introduction

Modern NoSQL systems frequently store documents as JSON objects without a rigid global schema. Within a single collection, different documents may contain different fields, nested objects, arrays, and type distributions. Queries such as the following are common:

```
user.age >= 30
user.name == "kim"
tags contains "backend"
orders[].price > 10000
profile.address.city exists
```

If the documents are unsorted and no auxiliary index exists, the worst-case cost of answering such predicates is a full scan over the document collection. The core performance question is therefore not whether the raw documents are sorted, but what auxiliary structures can map JSON predicates to small candidate sets without compromising correctness, write throughput, or operational manageability.

This paper proposes *SABLI*: Segmented Adaptive Bloom-LSM Inverted Index. SABLI is designed for collections containing millions or more JSON documents where three requirements must be satisfied simultaneously:

1. Support predicate search over arbitrary JSON paths, nested objects, arrays, equality, existence, containment, and range predicates.
2. Preserve write scalability through append-oriented storage, immutable segments, and background compaction.
3. Reduce cold-start and first-query latency by loading only metadata and lazily reading postings for relevant segments.

SABLI is not a direct replacement for LSM Trees or Bloom Filters. Instead, it separates concerns. The JSON inverted index determines *what* may match, the LSM-style segment lifecycle determines *how* data and index fragments are persisted and compacted, and Bloom Filters determine *where not to read*. This separation allows the system to combine the strengths of all three structures while controlling their individual weaknesses.

1.1 Contributions

The contributions of this design paper are as follows:

- A formal path-value extraction model for schema-less JSON documents, including scope identifiers for array-element semantics.
- A segmented inverted-index architecture that uses LSM-style mutable and immutable segments.
- A Bloom-pruned query execution model that eliminates impossible segments before posting-list access.
- An adaptive index-selection policy that chooses bitmaps, compressed postings, sparse postings, hash lookup, or range structures based on field cardinality and query type.
- A correctness argument showing that Bloom false positives cannot corrupt final results because final verification is mandatory.
- Complexity and implementation analysis for large collections such as those with five million or more documents.

2 Problem Definition

2.1 Document Model

Let the document collection be

$$\mathcal{D} = \{d_1, d_2, \dots, d_N\}. \quad (1)$$

Each document d_i is a finite JSON tree. Primitive values are drawn from

$$\mathcal{V} = \text{Null} \cup \text{Bool} \cup \text{Number} \cup \text{String}. \quad (2)$$

The set of normalized JSON paths is denoted by $\mathcal{P}$. Array positions may be represented either exactly, e.g., `orders[0].price`, or canonically using a wildcard token, e.g.,

$$\$.orders[]\text{.price}. \quad (3)$$

SABLI uses canonical array paths for broad containment search and scope identifiers for element-level consistency.

2.2 Query Model

A query Q is a Boolean composition of predicates. This paper focuses on conjunctive queries because they are the main case for candidate reduction:

$$Q = c_1 \wedge c_2 \wedge \dots \wedge c_k. \quad (4)$$

Each predicate c_i belongs to one of the following classes:

- equality: $(p = v)$,
- existence: $\text{exists}(p)$,
- containment: $(p[] \ni v)$,
- range: $(p \in [a, b])$, $(p \geq a)$, or $(p < b)$,
- scoped array predicate: multiple predicates that must hold within the same array element.

The exact answer is

$$\text{Ans}(Q) = \{d \in \mathcal{D} \mid d \models Q\}. \quad (5)$$

The objective is to compute $\text{Ans}(Q)$ without scanning all documents whenever selective predicates exist.

2.3 Operational Constraints

SABLI targets the following operational constraints:

- Raw documents do not need to be sorted by any query field.
- Writes should be append-oriented and crash recoverable.
- Indexes should be persistent and not rebuilt from raw JSON during normal startup.
- Updates and deletes should avoid immediate in-place mutation of large posting lists.
- Queries must be exact after final verification, even if intermediate filters admit false positives.

3 Background and Motivation

3.1 Path-Value Inverted Indexes

A JSON path-value index flattens a document into terms. For example:

```
{
  "user": { "name": "kim", "age": 31 },
  "tags": ["backend", "node"],
  "orders": [
    { "id": "A1", "price": 12000 },
    { "id": "A2", "price": 8000 }
  ]
}
```

can be transformed into

```
($.user.name, "kim", string, root)
($.user.age, 31, number, root)
($.tags[], "backend", string, $.tags[0])
($.tags[], "node", string, $.tags[1])
($.orders[].id, "A1", string, $.orders[0])
($.orders[].price, 12000, number, $.orders[0])
($.orders[].id, "A2", string, $.orders[1])
($.orders[].price, 8000, number, $.orders[1])
```

The posting list for a path-value term (p, v) is

$$I(p, v) = \{id(d) \mid (p, v, \tau, s) \in \text{Extract}(d)\}. \quad (6)$$

Such indexes are natural for JSON equality and containment queries. However, a naive design based on in-memory hash maps and sets has high memory overhead, poor cold-start behavior, expensive updates, and inefficient handling of high-cardinality fields such as UUIDs or email addresses.

3.2 LSM Trees

An LSM Tree maintains a write-ahead log, a mutable in-memory table, immutable sorted files, and background compaction [1]. It is effective for write-heavy workloads because writes are initially append-like and compaction is amortized. However, a basic LSM Tree organizes data around keys. Unless a suitable key or secondary index exists, it cannot efficiently answer ad hoc predicates over arbitrary JSON paths.

3.3 Bloom Filters

A Bloom Filter is a probabilistic membership structure [2]. For a set A and item x , its response satisfies:

$$x \in A \Rightarrow BF_A(x) = \text{true}, \quad (7)$$

$$x \notin A \Rightarrow BF_A(x) \in \{\text{false}, \text{true}\}. \quad (8)$$

There are no false negatives, but false positives are possible. Bloom Filters can reject absent terms with low memory overhead. They cannot return matching document identifiers and cannot answer range predicates directly. In SABLI, Bloom Filters are therefore used only as a pruning layer, never as an authority for final query answers.

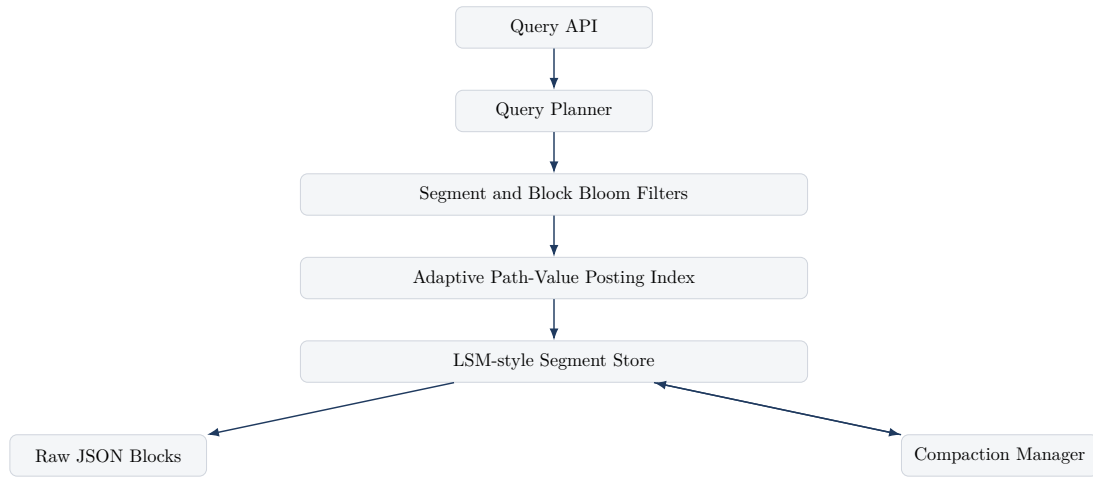


Figure 1: Layered architecture of SABLI. Bloom Filters prune access; adaptive postings produce candidates; raw-document verification produces final answers.

4 SABLI Architecture

SABLI combines the three structures as a layered engine.

4.1 Segment Layout

The collection is partitioned into immutable segments after data leaves the mutable write buffer. Each segment contains raw document blocks, dictionaries, postings, Bloom Filters, and deletion/version metadata.

```

segment-000001/
  manifest.json
  docs.zstd
  docs.offset
  path.dict
  value.dict
  shape.dict

  index/
    path_exists.idx
    value_posting.idx
    numeric_range.idx
    shape_signature.idx
    scope.idx

  bloom/
    segment.bloom
    path.bloom
    term.bloom
    block.bloom

  delete/
    delete.bitmap
    version.map
  
```

Only manifest files, dictionary headers, segment statistics, and Bloom Filters need to be loaded at startup. Posting lists and raw documents are loaded lazily.

4.2 Logical Components

SABLI consists of the following logical components:

1. **Write-Ahead Log**: records mutations before they are applied.
2. **Mutable MemSegment**: receives extracted terms and in-memory postings.
3. **Immutable Segments**: contain sealed raw document blocks and index fragments.
4. **Adaptive Posting Store**: stores each path or path-value index using a representation selected from cardinality and access statistics.
5. **Bloom Filter Layer**: rejects segments or blocks that cannot contain a predicate term.
6. **Compaction Manager**: merges segments, removes tombstones, discards old versions, and rewrites postings.
7. **Query Planner**: compiles predicates, orders them by estimated cardinality, intersects postings, and invokes verification.

5 Mathematical Model

5.1 Extraction Function

Definition 1 (JSON extraction). *For a document $d \in \mathcal{D}$, the extraction function is*

$$\text{Extract}(d) = \{(p, v, \tau, s) \mid p \in \mathcal{P}, v \in \mathcal{V}, \tau = \text{type}(v), s \in \mathcal{U}\}, \quad (9)$$

where s is a scope identifier. The root object has scope $\perp$, and each array element may induce a unique scope identifier.

Scope identifiers distinguish these two semantics:

- document-level semantics: a document contains one value somewhere and another value somewhere;
- array-element semantics: the same array element contains both values.

5.2 Term Encoding

A predicate term is encoded as

$$\kappa = \text{enc}(\text{pathId}, \text{typeId}, \text{valueId}). \quad (10)$$

The encoded term is used by Bloom Filters and equality postings. Range predicates use path-specific range structures rather than a single equality term.

5.3 Segmented Document Space

Let the segment set be

$$\mathcal{S} = \{S_1, S_2, \dots, S_m\}. \quad (11)$$

Each segment S_j owns a subset $\mathcal{D}_j \subseteq \mathcal{D}$, and

$$\mathcal{D} = \bigcup_{j=1}^m \mathcal{D}_j, \quad \mathcal{D}_a \cap \mathcal{D}_b = \emptyset \text{ for } a \neq b \quad (12)$$

for live physical versions before logical version filtering.

5.4 Posting Function

For a predicate c , the segment-local candidate posting is

$$P_j(c) = \{\text{id}(d) \mid d \in \mathcal{D}_j \text{ and } d \text{ satisfies the indexed candidate condition of } c\}. \quad (13)$$

For equality predicates this is the exact path-value posting before tombstone and version filtering. For range predicates this may be a bucket-derived superset.

5.5 Bloom Candidate Segments

For an equality-like predicate c , the Bloom candidate segment set is

$$M(c) = \{S_j \in \mathcal{S} \mid BF_j(c) = \text{true}\}. \quad (14)$$

For conjunctions,

$$M(Q) = \bigcap_{i=1}^k M(c_i). \quad (15)$$

If c_i is a range predicate, $M(c_i)$ may be computed from range min/max statistics, range Bloom variants, or conservatively set to all segments.

5.6 Candidate and Answer Sets

The candidate set for a conjunctive query is

$$\text{Cand}(Q) = \bigcup_{S_j \in M(Q)} \left(\bigcap_{i=1}^k P_j(c_i) \right). \quad (16)$$

Let T be the global set of deleted or hidden physical versions, and V be the map that selects the latest visible version for each logical document. The live candidate set is

$$\text{Live Cand}(Q) = V(\text{Cand}(Q) \setminus T). \quad (17)$$

The final answer is

$$\text{Ans}(Q) = \{d \in \text{Fetch}(\text{Live Cand}(Q)) \mid \text{Verify}(d, Q) = \text{true}\}. \quad (18)$$

6 Adaptive Index Selection

A central weakness of a naive inverted index is using the same representation for all fields. A low-cardinality field such as **status** benefits from compressed bitmaps, while a high-cardinality field such as **email** or **sessionId** may produce millions of single-item postings. SABLI therefore selects representations adaptively.

6.1 Cardinality Statistics

For each path p , SABLI maintains:

$$N_p = |\{d \in \mathcal{D} \mid \text{exists}_d(p)\}|, \quad (19)$$

$$C_p = |\{v \mid \exists d \in \mathcal{D} : (p, v, \tau, s) \in \text{Extract}(d)\}|, \quad (20)$$

$$\rho_p = \frac{C_p}{N_p}. \quad (21)$$

Here ρ_p is the cardinality ratio. It helps distinguish categorical fields from near-unique identifiers.

Field type	Example	Representation	Rationale
Low cardinality	status, country, role	Roaring bitmap	Dense repeated values compress well
Medium cardinality	tag, category, city	Delta-coded postings	Moderate list sizes with good compression
High cardinality	uuid, email, sessionId	Sparse posting or key lookup	Avoid many one-document bitmaps
Numeric range	age, price, timestamp	Range bucket + local tree	Range queries need ordered access
Array value	tags[], skills[]	Scoped posting	Supports containment and element semantics
Object shape	profile, order	Shape signature	Fast structural narrowing

Table 1: Adaptive index representation policy.

6.2 Representation Policy

The policy can be expressed as a decision function:

$$R(p) = f(N_p, C_p, \rho_p, \text{type}(p), \text{workload}(p)), \quad (22)$$

where $R(p)$ is the selected representation for path p .

7 Algorithms

7.1 Insertion

Insertion is append-oriented. The raw document and extracted terms are written to the mutable segment after a WAL record is persisted.

Listing 1: SABLI insertion algorithm

```

Algorithm SABLI_INSERT(d)
Input: JSON document d

1. id ← allocateDocId()
2. WAL.append(INSERT, id, d)
3. entries ← Extract(d)
4. DocStore.append(id, d)

5. for each (p, v, tau, s) in entries:
    pathId ← getOrCreatePathId(p)
    valueId ← getOrCreateValueId(v)
    termKey ← encode(pathId, tau, valueId)

    MemSegment.addPath(pathId, id)
    MemSegment.addTerm(termKey, id, s)
    MemSegment.updateStats(pathId, valueId, tau)
    MemSegment.updateBloom(termKey)

```

```

6. if MemSegment.size >= threshold:
    Flush(MemSegment)

```

The asymptotic CPU cost is proportional to the number of extracted JSON leaves.

7.2 Flush

Listing 2: Mutable segment flush

```

Algorithm SABLI_FLUSH(MemSegment M)

1. freeze M
2. sort or group postings by pathId and termKey
3. choose representation R(p) for each path p
4. encode postings and range structures
5. write raw document blocks and offset table
6. write dictionaries and segment statistics
7. write Bloom Filters
8. atomically publish immutable segment S
9. truncate WAL records covered by S

```

Flush is the boundary where the in-memory write representation is transformed into compact persistent index structures.

7.3 Search

Listing 3: SABLI search algorithm

```

Algorithm SABLI_SEARCH(Q)
Input: query Q

1. predicates <- Compile(Q)
2. candidateSegments <- all segments

3. for each predicate c in predicates:
    candidateSegments <- candidateSegments intersect SegmentMayContain(c)

4. predicates <- sortByEstimatedCardinality(predicates)
5. resultIds <- empty set

6. for each segment S in candidateSegments:
    local <- ALL_DOCS_IN_SEGMENT(S)

    for each predicate c in predicates:
        posting <- LoadPostingOrRange(S, c)
        local <- intersect(local, posting)

    if local is empty:
        break

    local <- local - S.deleteBitmap
    local <- removeOldVersions(local)
    resultIds <- union(resultIds, local)

7. docs <- FetchRawDocuments(resultIds)
8. return { d in docs | Verify(d, Q) = true }

```

The most selective predicate should be evaluated first. This reduces intermediate candidate sizes and lowers the number of postings that must be loaded.

7.4 Update

SABLI does not update old postings in place. An update is implemented as an append of a new physical version plus a tombstone for the old version.

Listing 4: SABLI update algorithm

```
Algorithm SABLI_UPDATE(logicalId, d_new)
1. oldPhysicalId <- VersionMap.lookup(logicalId)
2. newPhysicalId <- allocateDocId()
3. WAL.append(UPDATE, logicalId, oldPhysicalId, newPhysicalId, d_new)
4. mark oldPhysicalId in delete bitmap or mutable tombstone set
5. insert d_new as newPhysicalId
6. VersionMap[logicalId] <- newPhysicalId
```

This converts expensive random deletion from old postings into a cheap append and later compaction.

7.5 Delete

Listing 5: SABLI delete algorithm

```
Algorithm SABLI_DELETE(logicalId)
1. physicalId <- VersionMap.lookup(logicalId)
2. WAL.append(DELETE, logicalId, physicalId)
3. mark physicalId in delete bitmap or mutable tombstone set
4. remove logicalId from visible VersionMap
```

Queries subtract delete bitmaps before raw document verification.

7.6 Compaction

Listing 6: SABLI compaction algorithm

```
Algorithm SABLI_COMPACT(segments)
1. read selected immutable segments
2. discard deleted physical documents
3. discard old versions shadowed by newer versions
4. re-extract or merge surviving postings
5. recompute statistics and Bloom Filters
6. write new compacted segment
7. atomically replace old segments
```

Compaction controls long-term read amplification, removes tombstones, and improves index compression.

8 Correctness

8.1 Bloom Filters Do Not Affect Soundness

Lemma 1 (No false-negative pruning). *Assume each segment Bloom Filter contains every equality-like term present in the segment. If a document $d \in S_j$ satisfies an equality-like predicate*

c , then $S_j \in M(c)$.

Proof. If d satisfies c , the term for c occurs in segment S_j . By construction, the term was inserted into BF_j . Bloom Filters have no false negatives, so $BF_j(c) = \text{true}$. Hence $S_j \in M(c)$. $\square$

Corollary 1. *For a conjunctive query Q , Bloom pruning cannot remove a segment that contains a document satisfying all equality-like predicates of Q .*

8.2 Candidate Superset Property

Lemma 2 (Candidate superset). *If every segment posting $P_j(c)$ is exact or a superset of documents satisfying predicate c within S_j , then*

$$\{\text{id}(d) \mid d \models Q\} \subseteq \text{Cand}(Q) \quad (23)$$

before deletion and version filtering.

Proof. Let $d \models Q$. Then for every predicate c_i , d satisfies c_i . By the assumption on postings, $\text{id}(d) \in P_j(c_i)$ for the segment S_j containing d . By the no-false-negative property of Bloom pruning, $S_j \in M(Q)$ for equality-like predicates; range predicates are handled by conservative statistics or exact range structures. Therefore $\text{id}(d)$ appears in the intersection for S_j and hence in $\text{Cand}(Q)$. $\square$

8.3 Final Exactness

Theorem 1 (Exact answers). *If deletion/version metadata is correctly applied and $\text{Verify}(d, Q)$ implements the query semantics, then SABLI returns exactly $\text{Ans}(Q)$.*

Proof. First, by the candidate superset property, every live document satisfying Q is included in the live candidate set after deletion/version filtering. Thus no valid answer is lost. Second, SABLI returns only documents for which $\text{Verify}(d, Q) = \text{true}$. Therefore no document that fails the query semantics is returned. Hence the returned set is exactly $\text{Ans}(Q)$. $\square$

9 Complexity Analysis

Let

- N be the total number of documents,
- m be the number of segments,
- L be the average number of extracted JSON leaves per document,
- k be the number of predicates in the query,
- r be the number of segments remaining after Bloom and statistics pruning,
- P_i be the size of the selected posting for predicate c_i ,
- C be the number of candidate documents after posting intersection.

9.1 Insertion Cost

Extraction visits each primitive JSON leaf once. Therefore the insertion CPU cost is

$$O(L). \quad (24)$$

The disk path is append-oriented: WAL append plus document-block append. Flush and compaction costs are amortized.

9.2 Search Cost

Bloom checking over all segments for all predicates costs

$$O(km) \quad (25)$$

bit-probe operations. These operations are small and typically memory-resident. After pruning, only r segments are inspected. The posting processing cost is approximately

$$O\left(\sum_{S_j \in M(Q)} \text{cost}_{S_j}(P_1 \cap \dots \cap P_k) + C\right). \quad (26)$$

If predicates are ordered by ascending estimated cardinality, intermediate intersections are minimized.

9.3 Range Cost

For a range predicate, SABLI uses path-specific range buckets, local B-trees, or ordered block statistics. The cost is

$$O(B + C_b + C_v), \quad (27)$$

where B is the number of covered buckets or tree intervals, C_b is the number of boundary candidates, and C_v is the final verification cost.

9.4 Space Cost

Let T be the total number of extracted terms across all documents:

$$T = \sum_{d \in \mathcal{D}} |\text{Extract}(d)|. \quad (28)$$

The uncompressed posting-space lower bound is $O(T)$ docId references. SABLI reduces practical memory and disk cost through dictionary encoding, compressed postings, bitmaps for low-cardinality paths, sparse postings for high-cardinality paths, and lazy loading of segment-local indexes.

10 Comparison with Component Techniques

SABLI should be understood as a synthesis rather than a replacement. It uses LSM Tree mechanics to solve operational problems of persistent inverted indexes and uses Bloom Filters to reduce read amplification. The JSON path-value model remains responsible for semantic query execution.

11 Implementation Considerations

11.1 Cold Start

For large collections, the engine should not read raw JSON documents or decode all postings at startup. A practical startup sequence is:

1. load global manifest
2. load segment manifests
3. load path dictionary headers
4. load value dictionary headers or hot slices
5. load Bloom Filters and range min/max statistics
6. initialize LRU posting cache
7. accept queries

Property	Path-value index	LSM Tree	Bloom Filter	SABLI
JSON path search	Strong	Weak alone	Not supported	Strong
Compound predicates	Strong	Requires secondary indexes	Not supported	Strong
Write throughput	Medium	Strong	Not applicable	Strong
Update/delete handling	Weak if in-place	Strong with tombstones	Weak alone	Strong
Negative pruning	Limited	Optional	Strong	Strong
Cold-start behavior	Weak if memory-first	Mature	Strong	Strong
High-cardinality fields	Often weak	Strong as key lookup	Medium	Adaptive
Range query	Needs extra index	Strong for key ranges	Not supported	Range-aware
Exact result return	Yes	Yes	No	Yes

Table 2: Comparison of SABLI with its component techniques.

This design allows collections with millions of documents to start by loading metadata rather than the full data and index corpus.

11.2 Posting Cache

A posting cache should be keyed by

$$(\text{segmentId}, \text{pathId}, \text{predicateEncoding}). \quad (29)$$

Hot postings can remain decoded as Roaring bitmaps or sorted integer arrays. Cold postings remain compressed on disk.

11.3 Dictionary Encoding

Paths and values should be dictionary encoded:

$$\$.user.name \mapsto \text{pathId}, \quad \text{"backend"} \mapsto \text{valueId}. \quad (30)$$

This avoids repeated string storage and enables fixed-width term encodings.

11.4 Array Scope Semantics

For predicates analogous to MongoDB-style element matching, document-level intersection is insufficient. Consider:

```
orders[].id == "A1" AND orders[].price > 10000
```

The intended semantics may require the same order element to satisfy both predicates. SABLI stores postings of the form

$$(\text{docId}, \text{scopeId}) \quad (31)$$

for scoped paths. Query execution intersects on both *docId* and *scopeId* when element-level semantics are requested.

11.5 Compaction Policy

Compaction should balance three costs:

- write amplification from rewriting segment data,

- read amplification from too many small segments,
- space amplification from tombstones and obsolete document versions.

A tiered compaction policy favors high write throughput. A leveled policy can reduce read amplification. SABLI can choose either depending on workload.

12 Evaluation Plan

A complete implementation should be evaluated with both synthetic and production-like JSON corpora. The following metrics are essential:

- ingestion throughput in documents per second,
- cold-start time,
- equality query latency,
- range query latency,
- compound predicate latency,
- percentage of segments skipped by Bloom pruning,
- index size relative to raw JSON size,
- memory usage under posting-cache limits,
- update/delete throughput,
- compaction write amplification.

Suggested baselines include a raw full scan, a memory-first path-value index, an LSM key-value store with manually designed secondary indexes, and a search-engine-style inverted index. Queries should vary by selectivity, path depth, array usage, cardinality, and range width.

13 Limitations

SABLI controls but does not eliminate all trade-offs. Indexing every path and every value can still be expensive for highly heterogeneous collections. Complex nested-array semantics may require additional scope metadata and more expensive verification. Bloom Filter false positives can cause unnecessary reads, although they do not affect correctness. Range predicates require specialized structures and may produce candidates that need exact verification. Finally, compaction introduces write amplification and must be scheduled carefully to avoid interfering with foreground query latency.

14 Conclusion

SABLI is a unified algorithmic design for fast search over unsorted schema-less JSON documents. Its central principle is separation of responsibilities: adaptive path-value postings provide JSON query semantics, LSM-style segments provide write-optimized persistence and mutation management, and Bloom Filters provide low-cost negative pruning. By adding adaptive representation selection, scoped array postings, tombstone handling, lazy loading, and final raw-document verification, SABLI addresses the primary weaknesses of standalone path-value indexes, LSM Trees, and Bloom Filters.

The resulting architecture is suitable for large NoSQL-like document collections where full scans are unacceptable, raw-document ordering cannot be assumed, and both write scalability and expressive JSON predicate search are required.

References

- [1] P. O’Neil, E. Cheng, D. Gawlick, and E. O’Neil. The Log-Structured Merge-Tree (LSM-Tree). *Acta Informatica*, 33(4):351–385, 1996.
- [2] B. H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.
- [3] S. Chambi, D. Lemire, O. Kaser, and R. Godin. Better bitmap performance with Roaring bitmaps. *Software: Practice and Experience*, 46(5):709–719, 2016.
- [4] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber. Bigtable: A distributed storage system for structured data. *ACM Transactions on Computer Systems*, 26(2), 2008.
- [5] S. Ghemawat, H. Gobioff, and S.-T. Leung. The Google file system. *Proceedings of the 19th ACM Symposium on Operating Systems Principles*, 2003.